

HIGH LEVEL ARCHITECTURE

UNIVERSITY CORRIDOR MEETING ROOM AND WORKSPACE RESERVATION SYSTEM

TEAM: - SYED MEESUM RAZA NAQVI (24100183)

STUDENT ID	NAME
24100183	SYED MEESUM RAZA NAQVI

TABLE OF CONTENTS

1.	Introduction.....	3
2.	System Architecture.....	4
2.1	Architecture Diagram.....	4
2.2	Architecture Description.....	5
2.3	Justification of the Architecture	6
3.	Risk Management	7
3.1	Potential Risks and Mitigation Strategies	7
4.	Tools and Technologies	8
5.	Hardware Requirements.....	9
6.	Who Did What?	10
7.	Review checklist	10

1. Introduction

<Give an overview of the project here. The overview must highlight the overall objectives of the project and its potential users and customers.>

Overview:

The University Corridor Meeting Room and Workspace Reservation System makes it easier to book meeting rooms and individual workspaces in different university departments. Each department has several corridors, and each corridor contains multiple rooms. Some of these rooms have several workspaces to meet different needs. Workspaces are of different types too. The individual workspace for like studying alone, shared workspace for group studying or office hours. Other rooms may be reserved exclusively for faculty meetings and do not have multiple workspaces; instead, the entire room will be booked for a specified time. The system helps manage these spaces better, reduces booking conflicts, and makes sure the rooms are used efficiently.

Objectives:

- To provide a centralized platform for booking meeting rooms and workspaces.
- To allow users to view, reserve, and manage bookings efficiently.
- To prevent double bookings and enforce booking policies to maintain fair usage.

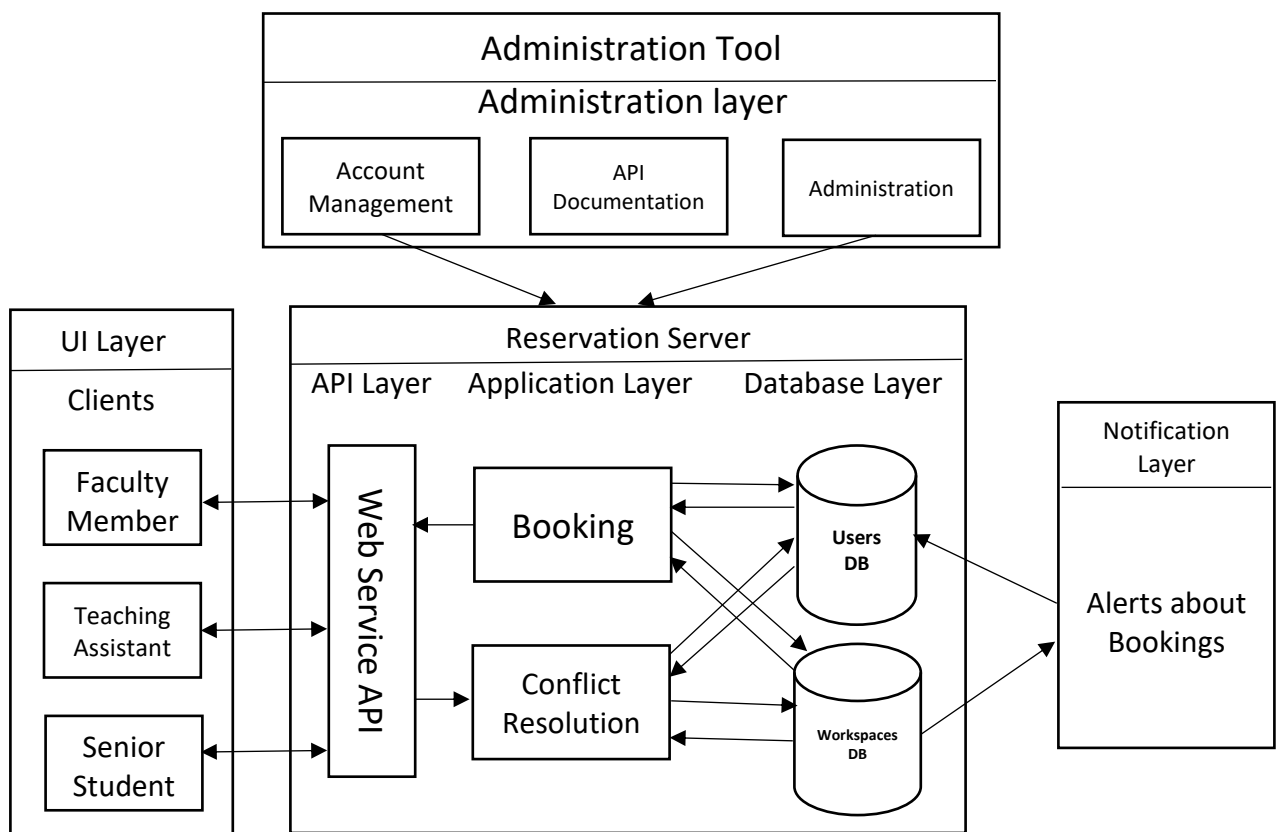
Potential Users:

- Faculty members for departmental meetings.
- Teaching Assistants (TAs) for office hours and student consultations.
- Senior Year Students for studying alone or in group for project (restricted to 5 members).
- Application system for handling booking, conflict resolution, and notification.
- Admin for creating user accounts based on the user's status.

2. System Architecture

2.1 Architecture Diagram

<Draw a diagram of the system architecture. Remember that your system's architecture may be defined using multiple architectural patterns.>



2.2 Architecture Description

<Give description of each subsystem in the architecture diagram above. Moreover, give description of how subsystems interact with each other>

- **User Interface (UI):** Users (faculty, TAs, students) interact with the system via a web application. They can log in, search for available rooms/workspaces, and make reservations.
- **Booking Module:** When a user attempts to book a room/workspace, the system checks the availability from the database. If the room is available, it is booked, and the details are updated in the database.
- **Conflict Resolution Module:** If multiple users try to book the same room simultaneously, the system applies conflict resolution rules (e.g., first-come, first-served) and updates the status accordingly.
- **Notification Service:** After booking, cancellation, or any change, the system sends a notification to the user's email or dashboard.

2.3 Justification of the Architecture

<List down pros and cons of the architecture you have defined in the context of your system. Moreover, give a justification of why this architecture is appropriate for your system. Make sure that you also discuss how this architecture helps in implementation of non-functional requirements. >

- **Pros:**
 - **Scalability:** The architecture can handle up to 500 concurrent users due to the layered design separating user interface, logic, and data storage.
 - **Reliability:** With separate modules for conflict resolution and notifications, the system can guarantee consistent handling of reservations, minimizing double bookings.
 - **Performance:** The booking system and conflict resolution module ensure responses are delivered within 2 seconds, meeting the performance requirements.
 - **Maintainability:** Modular design ensures that changes in one part of the system (e.g., UI or Notification Service) won't affect the rest of the system.
- **Cons:**
 - **Complexity:** The conflict resolution module adds complexity and requires precise handling to ensure smooth functioning during high traffic times.
 - **Notification Delays:** If the notification service fails, users may not receive timely updates, affecting user experience.

3. Risk Management

3.1 Potential Risks and Mitigation Strategies

<List down top 10 potential risks and their mitigation strategies>

Sr.	Risk Description	Mitigation Strategy
1.	Double bookings due to conflict resolution failure	Implementing robust testing for the conflict resolution module
2.	Notification service fails	Setting up retries and fallbacks for the notification service
3.	Unauthorized access to user data	Implementing role-based access control and encryption for data
4.	System crashes during peak usage times	Conducting stress testing and ensure proper load balancing
5.	Database connectivity issues	Implementing database replication and connection pooling
6.	Delays in response times	Optimizing code and use caching for frequently accessed data

7.	Data loss during server crashes	Implementing regular data backups and recovery mechanisms
8.	Security vulnerabilities	Performing regular security audits and implement patches
9.	Inaccurate booking data	Validating inputs and implement logging and rollback for errors
10.	System downtime during maintenance	Schedule maintenance during off-peak hours

4. Tools and Technologies

<List down tools and technologies that you plan to use for development and deployment. Make sure that you mention name and version of the tools.>

- **Front-End:** React.js (v18.0)
- **Back-End:** Node.js (v16), Express (v4.x)
- **Database:** MongoDB (v7.x)
- **Notification System:** NodeMailer for email notifications
- **Testing:** JUnit, Selenium for UI testing
- **Development IDE:** Visual Studio Code
- **Deployment Platform:** Vercel (v34.x) for hosting, S3 for storage
- **Version Control:** Git (GitHub repository)

5. Hardware Requirements

<List down the hardware requirements. This should include requirements for both development machines and deployment servers>

5.1 Development Machines

The development machines are required to run the full MERN stack locally for development purposes, including MongoDB for database management, Node.js for the backend, and React for the frontend. The specifications ensure smooth local development and testing.

- **Processor:** Intel Core i5 or equivalent (or higher)
- **RAM:** 8 GB minimum (16 GB recommended for optimal performance when running multiple services simultaneously)
- **Storage:** 256 GB SSD (or higher for faster data access and storage of local databases)
- **Operating System:** Windows 10, macOS, or Linux (Ubuntu 20.04 or later versions)
- **Software Requirements:**
 - Node.js (v16 or higher)
 - MongoDB (local installation or cloud-based like MongoDB Atlas for development)
 - Git (for version control)

5.2 Deployment Servers

Since Vercel is used for deployment, the infrastructure is largely managed by Vercel's serverless environment. Vercel handles scaling, RAM, and CPU allocations based on the needs of the system, which is built using Node.js for server-side functions and React for the frontend. For database needs, MongoDB Atlas will be preferred.

- **Processor:** Managed by Vercel's serverless infrastructure
- **RAM:** Managed by Vercel, typically sufficient for handling function executions (512MB–1GB per function)
- **Storage:** Vercel provides ephemeral storage for serverless functions (100MB per function)
- **Operating System:** Linux-based (managed by Vercel)
- **Database:** MongoDB Atlas (Cloud-based, managed by MongoDB for production and scalability) or MongoDB running locally (using Compass).

6. Who Did What?

Name of the Team Member	Tasks done
Syed Meesum Raza Naqvi	Project conception, requirements gathering, UML design, coding, testing, and report writing

7. Review checklist

Before submission of this deliverable, the team must perform an internal review. Each team member will review one or more sections of the deliverable.

Section Title	Reviewer Name(s)
Introduction	Syed Meesum Raza Naqvi
System Architecture	Syed Meesum Raza Naqvi
Risk Management	Syed Meesum Raza Naqvi
Tools and Technologies	Syed Meesum Raza Naqvi
Hardware Requirements	Syed Meesum Raza Naqvi